

LifeLink

SMART EMERGENCY RESPONSE SYSTEM

Project Documentation: QR Medical Profile, Accident Response and Alert Processing

GitHub Repository github.com/vinsikaboopathi/LifeLink
Public Profile Route LifeLink QR profile page
Project Purpose A prototype concept for sharing essential medical information and processing emergency alerts when an accident victim may be unable to communicate.

This document distinguishes current software prototype features from planned hardware and production features.

1. Abstract

LifeLink is a smart emergency-response concept intended to help responders access essential medical information when a road-accident victim may be unconscious or unable to communicate. The concept combines a QR-linked emergency profile, an accident-response confirmation countdown, location capture, a backend service and SMS notification processing. The current software prototype includes website pages for profile creation, QR access and an emergency countdown simulation, plus a Node.js/Express endpoint tested with Twilio. Physical sensor-based accident detection and full hardware integration remain planned work.

Privacy principle: The public QR profile should show only essential medical information. A personal mobile number must not be encoded in the QR or displayed on the public profile.

2. Problem Statement

After an accident, a victim may be unable to communicate their name, blood group, allergies or relevant medical conditions. Responders may also need a location and a reliable way to initiate an emergency notification. LifeLink proposes a QR profile for essential information and a confirmation-based alert workflow to reduce accidental notifications.

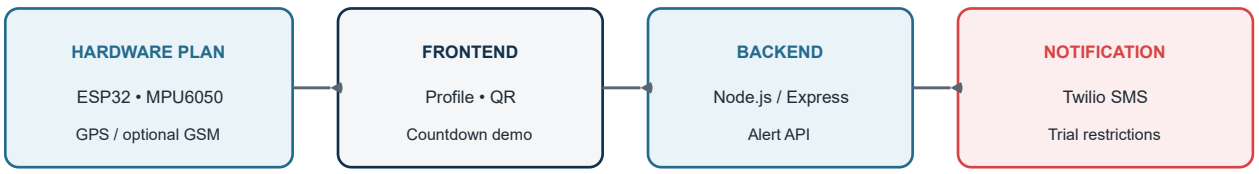
3. Objectives

- Provide QR access to a profile with name, blood group, allergies and medical condition.
- Demonstrate a 30-second countdown before emergency mode is triggered.
- Define a backend path for sending emergency status and location data.
- Support SMS processing when the SMS account, connectivity and configuration allow it.
- Keep personal mobile numbers private from the public QR profile.
- Create a clear integration path to ESP32-based hardware.

4. Proposed System

The planned system has three layers: a device layer for sensing and location, a software layer for profile and emergency workflows, and a notification layer for alert delivery. In the planned hardware flow, a possible accident starts a confirmation countdown. If the user does not cancel, the system obtains location data where available and passes an emergency request to the backend.

5. System Architecture



LAYER	COMPONENTS	ROLE

Hardware	ESP32, MPU6050, GPS, optional GSM	Planned: sense impact/tilt, obtain coordinates and start the response flow.
Frontend	HTML, CSS, JavaScript	Profile creation, QR page and emergency countdown simulation.
Backend	Node.js, Express	Receive emergency requests and ask the SMS provider to deliver a message.
Notification	Twilio SMS	SMS delivery subject to trial-account permissions and configuration.

Planned accident-response flow

MPU6050 event → ESP32 evaluation → 30-second confirmation → GPS coordinates → backend API → emergency processing → SMS gateway → designated recipient

QR information flow

QR scan → public LifeLink profile → name, blood group, allergies and medical condition. The personal mobile number should remain private and must not be included in the QR payload.

6. Hardware Requirements (Planned)

- **ESP32:** main controller for sensor readings and device workflow.
- **MPU6050:** accelerometer/gyroscope for sudden impact or abnormal tilt detection; thresholds require calibration.

- **GPS module:** provides latitude and longitude where satellite reception is available.
- **GSM module (optional):** can send SMS over a cellular network without a cloud backend, if cellular coverage and SIM service are available.
- **Power, mounting and enclosure:** needed for stable operation and consistent sensor readings.

These components describe the planned hardware design. Do not claim physical accident detection is implemented unless the hardware has been assembled and tested.

7. Software Components

COMPONENT	PURPOSE	STATUS
HTML/CSS/JavaScript	Website pages, profile form, QR and emergency simulator.	Implemented in repository.
Browser localStorage	Stores profile data in the current browser.	Implemented; not a shared database.
Node.js/Express	Backend endpoint for emergency SMS requests.	Endpoint tested.

Twilio	SMS gateway.	Predefined trial SMS tested; custom dynamic message restricted by trial account.
Database	Persistent profiles and private numbers.	Future enhancement; not currently implemented.

Emergency Workflow

- **Step 1 — Event:** In the planned hardware version, sensor readings indicate a possible accident.
- **Step 2 — Confirmation:** A 30-second countdown gives the user a chance to cancel a false alarm.
- **Step 3 — Emergency mode:** If not cancelled, the workflow marks the event as an emergency.
- **Step 4 — Location:** GPS coordinates are obtained where a GPS source is available; the website demo may use browser location or a demonstration fallback.
- **Step 5 — Backend:** Emergency details are sent to the Node.js endpoint.
- **Step 6 — Notification:** The backend requests SMS delivery. Dynamic delivery depends on provider permissions and configuration.

8. Current Prototype Status

FEATURE	STATUS	DETAIL
Website pages	Implemented	Home, profile creation, QR and emergency simulation pages exist in the repository.
Profile storage	Local browser	Uses localStorage; data does not automatically sync across devices.

QR route	Present	QR points to the public profile URL. Cross-device profile data must be served independently of localStorage.
Countdown	Tested locally	If deployed countdown freezes, deployed files/cache may differ from the local version.
Backend SMS request	Endpoint tested	Twilio trial rejected a custom dynamic message because only predefined templates are allowed.
Predefined trial SMS	Delivery tested	A predefined template test SMS was received successfully.
Physical sensors/GPS/GSM	Planned	Do not claim physical accident detection or direct hardware SMS is complete until tested.

9. Privacy and Safety

- The public QR profile should expose only name, blood group, allergies and medical condition.
- Personal mobile number should not appear on the QR or public profile.
- localStorage is browser-specific and is not suitable as secure multi-user production storage.
- A production backend should protect private numbers, validate API requests and use HTTPS.
- Sensor thresholds need testing against normal braking, potholes, device drops and other false-positive events.
- This is a prototype concept, not a certified emergency dispatch or medical system.

10. Testing Plan

- Verify profile creation, LifeLink ID display and profile editing.
- Confirm the personal mobile number is not visible on the public QR profile.
- Scan the QR using a second device and check what data is visible.
- Test countdown, cancel action, timeout behaviour and location button.
- Test backend requests and verify success/failure responses accurately.
- Test SMS only with permitted templates; report delivery only when confirmed by the provider.
- For hardware, test sensor calibration, GPS reception, cellular signal and power stability.

11. Future Enhancements

- Integrate ESP32, MPU6050 and GPS and calibrate accident-detection logic.
- Add a secure backend database for profiles and private emergency numbers.
- Configure a production-capable SMS workflow and verified recipient handling.
- Add event logs, timestamps, retries and failure handling.
- Improve QR durability with protected labels/engraving and possible redundant placements.
- Run controlled field tests and evaluate false positives/negatives before real-world use.

12. Conclusion

LifeLink demonstrates a software prototype for QR-based emergency information access and an accident-response countdown workflow, with a backend SMS request path tested within Twilio trial limitations. The planned ESP32, MPU6050, GPS and optional GSM integration provides a route

toward a physical IoT prototype. Key next steps are secure shared profile storage, hardware integration, reliable alert delivery and safety testing.

Presentation note: Describe website and backend functions as prototype features. Describe physical accident detection, persistent database storage and unrestricted dynamic SMS delivery as planned work until implemented and verified.

GitHub repository: https://github.com/vinsikaboopathi/LifeLink

Public profile route: https://vinsikaboopathi.github.io/LifeLink/profile.html

Prepared from project details shared in this conversation; hardware completion and production deployment have not been independently verified.